VELLORE INSTITUTE OF TECHNOLOGY
School of Computer Science & Engineering

SWAYALANG

A Domain-Specific Language, Optimizing Compiler & Execution Engine for Backtestable Stock Trading Strategies

Course: BCSE307P — Compiler Design Laboratory
Project Phase: Phase 3 — Final Implementation, Optimization & Testing
Evaluation: Review 3 — Final Project Review
Author: Swayam Shukla
Registration Number: 24BCE0751
Faculty / Lab Instructor: Department of Computer Science & Engineering
Date of Submission: October 2026
Repository Status: Production Ready (84/84 Automated Tests Passing, 100% Pass Rate)

Phase 1: Lexical, Recursive-Descent Parser & Typed AST	**PASSED / 100%**
Phase 2: Scoped Symbol Table, TAC Quadruples & Backtest Engine	**PASSED / 100%**
Phase 3: Multi-Pass TAC Optimizer, Risk Profiler & Web IDE	**PASSED / 100%**
Overall Test Suite (11 Test Modules in Pytest)	**84 / 84 Passed (100%)**

Executive Summary

SwayaLang is a specialized Domain-Specific Language (DSL), multi-pass optimizing compiler, and vector backtesting execution engine designed to express, optimize, and simulate quantitative stock trading strategies.

In traditional quantitative finance, trading strategies are commonly implemented in general-purpose scripting languages (such as Python or R). While flexible, this approach frequently introduces subtle yet catastrophic failures: lookahead bias (peeking into future price bars), dynamic runtime type errors in long-running backtests, high execution boilerplate, and unoptimized redundant computation across complex indicator conditions.

SwayaLang resolves these domain challenges by treating trading strategies as formal, compilable programs. The system features:

1. **Front-End Compiler Pipeline:** A regular-expression deterministic lexical scanner (`compiler/lexer.py`), an LL(1) recursive-descent syntax parser producing an immutable, typed Abstract Syntax Tree (`compiler/parser.py`, `compiler/ast_nodes.py`), and a hierarchical scoped symbol table (`compiler/symbol_table.py`) with strict semantic safety traps (`compiler/semantic.py`).
2. **Intermediate Code Generation:** Translation of strategy logic into a machine-independent Three-Address Code (TAC) Quadruple representation (`compiler/ir.py`).
3. **Phase 3 Multi-Pass Optimization Engine (`compiler/optimizer.py`):** An optimizing compiler pass implementing compile-time Constant Folding, Algebraic Identity Simplification, Boolean Short-Circuiting, Strategy-Wide Common Subexpression Elimination (CSE via value numbering for pure indicators), and Dead Code Elimination.
4. **Execution & Risk Engine (`compiler/interpreter.py`, `compiler/reporter.py`):** A lookahead-free vector indicator engine and discrete-event portfolio state machine producing institutional-grade financial risk profiles (Sharpe, Sortino, Maximum Drawdown %, CAGR %, Profit Factor) and dark-mode equity curve visualizations.
5. **Interactive Developer Tooling:** Dual-mode user interfaces consisting of a feature-complete Command-Line Interface (`driver.py`) and a modern 7-tab single-page Web IDE (`app.py`).

The entire compiler has been validated across an automated test suite of **84 unit and integration test cases** with a **100% pass rate**, demonstrating rigorous software architecture, algorithmic efficiency, and formal compiler construction principles.

1. Problem Statement & Objectives

1.1 Motivation

Algorithmic trading strategies rely on conditional execution rules evaluated over historical time-series data. However, script-based backtesting engines suffer from critical software engineering and domain deficiencies:

- **Lookahead Bias & Data Snooping:** Accidental future bar leakage (e.g., accessing bar t+1 data while evaluating conditions on bar t) invalidates historical simulations.
- **Dynamic Type Fragility:** Scripting runtimes fail to catch type mismatches, division-by-zero, negative indicator lookbacks, or uninitialized state variables until hours into a simulation.
- **Redundant Calculation Overheads:** Multi-rule strategies frequently re-evaluate identical moving average crossovers or RSI thresholds within the same price bar, incurring high compute overhead.
- **Coupled Infrastructure:** Strategy logic is typically intertwined with broker APIs, execution handlers, and charting libraries, preventing static analysis and intermediate representation transformations.

1.2 Project Objectives

1. **Formal Language Specification:** Design an expressive, declarative EBNF grammar tailored for quantitative trading rules.

2. **Full-Featured Compiler Pipeline:** Build from scratch (without compiler generators like Lex/Yacc) a lexical scanner, recursive-descent parser, scoped symbol table, semantic checker, and TAC generator.
3. **Machine-Independent Intermediate Optimization:** Implement multi-pass optimizations (constant folding, algebraic simplification, CSE, dead-code removal) directly on TAC quadruples.
4. **Causal Simulation Runtime:** Execute strategies bar-by-bar with zero lookahead bias, ensuring mathematical correctness and historical isolation.
5. **Quantitative Financial Reporting:** Compute standard financial risk metrics and produce publication-quality visualizations.
6. **Empirical Benchmarking & Testing:** Validate system performance across dataset scales (100 to 10,000 bars) and ensure complete test coverage.

---

2. Review 3 Deliverables & Syllabus Rubric Alignment

Review 3 Rubric Criteria	Weightage / Focus	SwayaLang Implementation Evidence
Project Completion	100% Core Scope	All 8 planned compiler stages, optimizer, reporter, web IDE, and CLI fully implemented.
Functional Correctness	Verification & Truth	**84/84 automated tests passing**; bitwise trade equivalence verified between unoptimized and optimized code.
Technical Complexity	Depth of Algorithms	Value-numbering CSE, multi-pass constant folding, vector indicators, portfolio state machine, microsecond latency profiling.
Compiler Integration	Theory to Practice	Lexing → Recursive Descent → Typed AST → Scoped Symbol Table → TAC Quadruples → Multi-pass Optimizer → Interpreter.
Innovation & Domain Fit	Originality	First-of-its-kind DSL preventing lookahead bias at the language and compiler level with automated Wall-Street risk profiling.
Code Quality	Clean Architecture	Strict PEP 8 standards, static type annotations, frozen immutable dataclasses, zero global mutable state.
Testing	Comprehensive Coverage	Unit, integration, negative, boundary, capital scaling, and CLI end-to-end driver tests (`tests/`).
Demonstration	Working Systems	Interactive 9-step automated lab evaluation script (`driver.py --demo3`) and 7-tab Web IDE (`app.py`).
Documentation	Academic Quality	Exhaustive 15-chapter IEEE-style report, 18-slide Keynote presentation deck, and comprehensive User Guide.

3. System Architecture

SwayaLang enforces a strict unidirectional data-flow pipeline. Each compiler subsystem consumes exclusively the structured artifact produced by the preceding stage, ensuring high cohesion, loose coupling, and modular verification.

3.1 Architectural Pipeline Overview

```
[ Source File: .strat ] ■ ▼ Stage 1: Lexical Analysis
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Lexer (Regex DFA
Tokenizer, Line/Col Coordinates) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Token Stream:
List[Token] ▼ Stage 2: Syntax Analysis
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Recursive-Descent Parser
& AST Construction ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
Abstract Syntax Tree: ProgramNode (Frozen Dataclasses) ▼ Stage 3 & 4: Semantic Analysis
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Scoped Symbol Table &
Semantic Safety Checker ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Validated, Scoped Annotated AST ▼ Stage 5: Intermediate Code Generation
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Three-Address Code (TAC)
Quadruple Generator ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
Linearized TAC Quadruples: List[Quadruple] ▼ Stage 6: Intermediate Code Optimization (Phase 3 Core)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Multi-Pass Optimizer
(Fold, Identities, CSE, DCE) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Optimized TAC Quadruples
& Folded AST ▼ Stage 7: Execution & Simulation Runtime
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Vector Indicator
Precomputation & Bar-by-Bar Simulator ■ ■■■ [ Historical OHLCV CSV ]
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Execution Log, Trade
Ledger & Equity Curve ▼ Stage 8: Financial Analytics & System Profiling
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Quantitative Risk
Profiler, Benchmark & Exporters ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■ Financial Metrics:
Sharpe, Sortino, Drawdown %, CAGR % ■■■ High-DPI Charts: Equity Curve & Underwater Drawdown PNG ■■■
Structured Exports: JSON & CSV Trade Logs
```

3.2 Architectural Subsystems Specification

Subsystem Tier	Module Path	Input Artifact	Output Artifact	Theoretical Compiler Concept
Front-End	`compiler/lexer.py`	Strategy Source Text (`.strat`)	`List[Token]`	Regular Expressions, Maximal Munch DFA
Front-End	`compiler/parser.py`	`List[Token]` Stream	`ProgramNode` AST	Recursive-Descent, LL(1) Deterministic Grammar
Front-End	`compiler/ast_nodes.py`	Parser Method Invocations	Immutable AST Hierarchy	Frozen Typed Dataclasses, Visitor Pattern
Middle-End	`compiler/symbol_table.py`	Identifier Declarations	Scoped Environment Tree	Hierarchical Lexical Scoping, Name Resolution
Middle-End	`compiler/semantic.py`	`ProgramNode` + `SymbolTable`	Annotated AST / Diagnostics	Static Type Checking, Arity/Safety Traps
Middle-End	`compiler/ir.py`	Validated AST	`List[Quadruple]` (TAC)	Linearized SSA-like Form, Synthetic Temporaries
Middle-End	`compiler/optimizer.py`	Raw `List[Quadruple]` + AST	Optimized Quadruples + AST	Constant Folding, Algebraic Laws, Value-Numbering CSE
Back-End	`compiler/interpreter.py`	Optimized TAC / AST + CSV	Trade Ledger + Portfolio State	Vector Indicators, Discrete-Event State Machine
Back-End	`compiler/reporter.py`	Portfolio History + Timers	Metrics + Plots + JSON/CSV	Risk Analysis, Benchmarking, Microsecond Tracing
Tooling	`driver.py`	CLI Arguments & Flags	Terminal Output & Files	Unified Compiler Command-Line Driver
Tooling	`app.py` + `frontend/`	HTTP Requests	Interactive Web IDE	Single-Page Application, Real-Time Optimization Diffs

3.3 Data Lowering Transformation Progression

```
[Source Text] ■■■■■ [Token Stream] ■■■■■■■■■ [AST Tree] ■■■■■■■■■■ [TAC Quadruples] ■■■■
[Optimized TAC] ■■■■ [Trade Records] ■■■■ [Risk Metrics] STRATEGY "..." TOKEN(KEYWORD, 'BUY')
StrategyNode() (ADD, t0, t1, t2) (ADD, 50, 20, 70) Trade(BUY, 100) Sharpe: 1.84 capital 100000
TOKEN(NUMBER, '100') ActionNode(BUY, 100) (GT, sma, 200, t3) (GT, sma, 200, t3) PnL: +$1,240.50
Drawdown: 4.2%
```

---

4. Formal Grammar Specification (EBNF)

SwayaLang is defined by an unambiguous, non-left-recursive formal grammar parseable by a single-token lookahead LL(1) parser.

4.1 Production Rules

```
program ::= { strategy_def } EOF strategy_def ::= 'STRATEGY' STRING_LIT '{' strategy_body '}'
strategy_body ::= { indicator_decl } rule_block { rule_block } indicator_decl ::= 'INDICATOR' IDENT '='
func_call func_call ::= IDENT '(' arg_list ')' arg_list ::= arg { ',' arg } arg ::= price_field |
NUMBER_LIT | IDENT price_field ::= 'CLOSE' | 'OPEN' | 'HIGH' | 'LOW' | 'VOLUME' | 'PRICE' | 'ENTRY_PRICE'
rule_block ::= rule_keyword ':' action 'IF' condition rule_keyword ::= 'ENTRY' | 'EXIT' | 'STOP_LOSS' |
'TAKE_PROFIT' action ::= buy_action | sell_action | short_action buy_action ::= 'BUY' ( order_qty |
'ALL' ) [ 'OF' STRING_LIT ] sell_action ::= 'SELL' ( order_qty | 'ALL' ) [ 'OF' STRING_LIT ] short_action
::= 'SHORT' order_qty [ 'OF' STRING_LIT ] order_qty ::= NUMBER_LIT 'SHARES' condition ::= or_condition
or_condition ::= and_condition { 'OR' and_condition } and_condition ::= primary_cond { 'AND'
primary_cond } primary_cond ::= '(' condition ')' | comparison comparison ::= expr comparator expr
comparator ::= '>' | '<' | '>=' | '<=' | '==' expr ::= term { ( '+' | '-' ) term } term ::= factor { (
'*' | '/' ) factor } factor ::= '(' expr ')' | NUMBER_LIT | price_field | IDENT
```

4.2 Operator Precedence and Associativity

The parser resolves expression ambiguity using standard precedence climbing implemented via recursive-descent helper methods:

Precedence Level	Operators	Associativity	Grammar Production	
1 (Highest)	Parentheses **`(...)`**	Non-associative	**`factor ::= '(' expr ')'`**	
2	Multiplicative Arithmetic: **`*`**, **`/`**	Left-to-Right	`term ::= factor { ('*' \	'/') factor }`
3	Additive Arithmetic: **`+`**, **`-`**	Left-to-Right	`expr ::= term { ('+' \	'-') term }`
4	Relational Comparators: **`>`**, **`<`**, **`>=`**, **`<=`**, **`==`**	Non-associative	**`comparison ::= expr comparator expr`**	
5	Logical Conjunction: **`AND`**	Left-to-Right	**`and_condition ::= primary_cond { 'AND' primary_cond }`**	
6 (Lowest)	Logical Disjunction: **`OR`**	Left-to-Right	**`or_condition ::= and_condition { 'OR' and_condition }`**	

4.3 Formal LL(1) Parsing Properties

1. **Absence of Left-Recursion:** All non-terminals derive productions starting with either terminal symbols or strictly higher-precedence non-terminals (A → β A'), guaranteeing termination without infinite descent.
2. **Pairwise Disjoint Lookahead Sets:** For every production with alternatives A → α_1 \mid α_2 \mid ... \mid α_k, the set FIRST(α_i) ∩ FIRST(α_j) = ∅ for all i ≠ j. Specifically:

- Rule keywords: **`ENTRY`**, **`EXIT`**, **`STOP_LOSS`**, and **`TAKE_PROFIT`** have mutually disjoint single-token lookaheads.
- Action verbs: **`BUY`**, **`SELL`**, and **`SHORT`** are unique single tokens.
- Comparators: **`>`**, **`<`**, **`>=`**, **`<=`**, and **`==`** are pairwise distinct terminal tokens.

3. **Linear Time Determinism:** Parsing is guaranteed to execute in strict O(N) time, where N is the number of lexical tokens.

5. Front-End Compiler Implementation

5.1 Lexical Analysis (`compiler/lexer.py`)

The lexical scanner converts raw UTF-8 source code into a sequential stream of `Token` instances.

- **Specification:** Uses ordered regular expression patterns compiled with `re.VERBOSE`.
- **Source Coordinates:** Tracks line numbers and 1-indexed column offsets dynamically across newline delimiters for precise diagnostics.
- **Comment Stripping:** Discards single-line comments prefixed by `#` without contaminating the token stream.
- **Lexical Traps:** Traps unexpected characters (e.g., `@`, `` `) and raises `` LexError` immediately.

5.2 Syntax Analysis & AST Lowering (`compiler/parser.py`, `compiler/ast_nodes.py`)

The parser consumes tokens via `match()` and `consume()` methods, constructing an immutable tree of dataclasses.

- **Node Immutability:** All AST nodes are marked `@dataclass(frozen=True)`, preventing accidental in-place mutation during compilation passes.
- **Node Hierarchy:**
- `ProgramNode`: Top-level container holding a list of `StrategyNode` instances.
- `StrategyNode`: Encloses strategy identifier, indicator declarations, and rule blocks.
- `IndicatorDeclNode`: Pairs indicator symbol with a `FuncCallNode`.
- `RuleBlockNode`: Encapsulates rule kind (`ENTRY`, `EXIT`, etc.), `ActionNode`, and condition AST.
- `BinaryOpNode` / `ComparisonNode`: Represents binary arithmetic and boolean logic trees.

---

6. Symbol Table & Semantic Analysis

6.1 Scoped Symbol Table (`compiler/symbol_table.py`)

SwayaLang implements a hierarchical scoped symbol table:

- **Global Scope:** Pre-populated with built-in indicator functions (`SMA`, `EMA`, `RSI`), allowable price fields (`CLOSE`, `OPEN`, `HIGH`, `LOW`, `VOLUME`), and contextual built-ins (`PRICE`, `ENTRY_PRICE`).
- **Strategy Child Scopes:** Each strategy block instantiates a child scope inheriting from the global table. User-defined indicators are registered locally, ensuring complete lexical isolation between independent strategies defined in the same source file.

6.2 Semantic Analysis & Safety Traps (`compiler/semantic.py`)

The semantic analyzer traverses the AST and enforces domain invariants:

1. **Duplicate Identifier Detection:** Flags redeclaration of existing indicators or duplicate strategy names.
2. **Indicator Arity & Parameter Bounds:** Enforces that `SMA`, `EMA`, and `RSI` receive exactly 2 arguments: a valid price field and a positive integer period (1 ≤ period ≤ 500).
3. **Domain Entry Traps:** Forbids referencing `ENTRY_PRICE` inside an `ENTRY` rule block, as entry price is undefined prior to position establishment.
4. **Static Arithmetic Safety:** Detects static division by zero (e.g., `close / 0`) at compile time.
5. **Mandatory Entry Rule:** Guarantees that every strategy declares at least one `ENTRY` block.

7. Intermediate Representation: Three-Address Code

7.1 Quadruple Format (`compiler/ir.py`)

To decouple high-level grammar from low-level execution and enable machine-independent optimization, the IR generator translates the AST into a linear sequence of Three-Address Code (TAC) Quadruples.

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Three-Address Code Quadruple ■ ■ ■ ■ ( Operator, Argument 1, Argument 2, Result ) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Quadruple Field	Type	Description
`op`	**`str`**	Opcode (e.g., **`ADD`**, **`SUB`**, **`MUL`**, **`DIV`**, **`GT`**, **`LT`**, **`AND`**, **`OR`**, **`LABEL`**, **`IF_FALSE`**, **`ACTION`**)
`arg1`	**`Any`**	First operand (literal number, variable name, or temporary register **`t0, t1, ...`**)
`arg2`	**`Any`**	Second operand (or **`None`** for unary operations and jumps)
`result`	**`str`**	Destination temporary register, control-flow label, or execution target

7.2 Synthetic Temporary & Control-Flow Generation

- **Temporary Register Allocation:** Allocates monotonically increasing synthetic registers (**`t0, t1, t2, ...`**).
- **Control Labels:** Emits branch labels (**`L0, L1, L2, ...`**) for conditional evaluation.
- **Short-Circuit Emission:** Emits **`IF_FALSE <cond> GOTO <exit_label>`** instructions to bypass unneeded computations when rule conditions are not met.

8. Code Optimization (Phase 3 Core Deliverable)

8.1 Multi-Pass Optimizer Architecture (`compiler/optimizer.py`)

Phase 3 introduces a multi-pass optimizing engine that operates across both AST and TAC representations.

```
Raw Quadruples ■■■ [Pass 1: Constant Folding] ■■■ [Pass 2: Algebraic Laws] ■■■ [Pass 3: CSE] ■■■
[Pass 4: DCE & Jumps] ■■■ Optimized TAC
```

8.2 Pass 1: Constant Folding & Constant Propagation

Static arithmetic expressions containing literal numeric constants are evaluated at compile time rather than repeatedly at every historical price bar.

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Constant Folding Rule: ■ ■ ■ ■ ENTRY_PRICE * (1.0 - 0.05) ■■■ ENTRY_PRICE * 0.95 ■ ■ 10 + 20 ■■■
30 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

8.3 Pass 2: Algebraic Identity Simplification

Eliminates identity and absorbing operations across arithmetic and boolean logic:

Algebraic Rule	Mathematical Identity	Transformed Representation
Multiplication by One	x × 1.0 = x	x
Addition of Zero	x + 0.0 = x	x
Subtraction of Zero	x - 0.0 = x	x
Multiplication by Zero	x × 0.0 = 0.0	0.0
Division by One	x ÷ 1.0 = x	x
Boolean AND with True	True ∧ P = P	P
Boolean AND with False	False ∧ P = False	False
Boolean OR with True	True ∨ P = True	True
Boolean OR with False	False ∨ P = P	P

8.4 Pass 3: Common Subexpression Elimination (CSE)

SwayaLang implements value numbering for Common Subexpression Elimination across strategy rules:

- **Global Strategy Invariant:** Market indicators (`SMA`, `EMA`, `RSI`) and historical price fields are pure, read-only time-series functions. Their values remain constant across all rule evaluations within a given price bar.
- **Expression Hashing:** Subexpressions are hashed into a value table:

Key = hash(op, arg1, arg2)

- **Commutative Normalization:** Commutative operators (`+`, `*`, `==`, `AND`, `OR`) are normalized by sorting arguments lexicographically: min(arg1, arg2), max(arg1, arg2).
- **Redundancy Elimination:** Subsequent identical calculations reuse the previously assigned temporary register, eliminating redundant computation.

8.5 Pass 4: Dead Code Elimination (DCE) & Jump Optimization

- **Temporary Pruning:** Any temporary register that is defined but never referenced downstream in an active action, condition, or jump is eliminated.
- **Fall-Through Jump Pruning:** Conditional jumps targeting the immediately following instruction (`IF_FALSE t GOTO L1; LABEL L1`) are recognized as no-ops and purged.

8.6 Optimization Reduction Metric

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Instruction Reduction Percentage: ■ ■ ■ ■ Reduction % = ( ( Count_unoptimized - Count_optimized )
■ ■ / Count_unoptimized ) * 100% ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

On representative strategies (such as `multi_rule.strat`), the optimizer achieves **up to 28.6% instruction reduction** while preserving exact bitwise simulation equivalence.

9. Execution Engine & Vector Backtest Runtime

9.1 Vector Technical Indicator Formulations (`compiler/interpreter.py`)

To prevent nested loops over historical data, indicators are precomputed using vectorized causal sliding windows.

A. Simple Moving Average (SMA)

Computes the arithmetic mean of closing prices over a rolling lookback window of N periods:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Simple Moving Average (SMA): ■ ■ ■ ■ 1 N-1 ■ ■ SMA_t(N) = ■■■ * Σ Close_{t-i} ■ ■ N i=0 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Parameter	Symbol	Definition
Lookback Period	N	Number of price bars in moving window (1 ≤ N ≤ 500)
Current Bar Index	t	Discrete time index of the historical simulation
Closing Price	Close_t	Official settlement price at bar t

B. Exponential Moving Average (EMA)

Applies exponentially decaying weighting to historical prices, assigning greater significance to recent bars:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Exponential Moving Average (EMA): ■ ■ ■ ■ Smoothing Multiplier: α = 2 / ( N + 1 ) ■ ■ ■ ■
Recursive Formula: EMA_t(N) = α * Close_t + (1 - α) * EMA_{t-1} ■ ■ Initial Seed: EMA_0(N) = Close_0
■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

C. Relative Strength Index (RSI - Wilder's Smoothing)

Measures the velocity and magnitude of directional price movements on a normalized scale [0, 100]:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Relative Strength Index (RSI - Wilder's Smoothed): ■ ■ ■ ■ 1. Bar-to-Bar Change: ΔP_t = Close_t -
Close_{t-1} ■ ■ 2. Directional Movements: U_t = max( ΔP_t, 0 ) ■ ■ D_t = max( -ΔP_t, 0 ) ■ ■ 3.
Smoothed Moving Averages: ■ ■ AvgGain_t = ( AvgGain_{t-1} * (N - 1) + U_t ) / N ■ ■ AvgLoss_t = (
AvgLoss_{t-1} * (N - 1) + D_t ) / N ■ ■ 4. Relative Strength: RS_t = AvgGain_t / AvgLoss_t ■ ■ 5.
Normalized RSI Index: ■ ■ RSI_t = 100 - ( 100 / ( 1 + RS_t ) ) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

9.2 Portfolio State Machine

The runtime iterates sequentially through historical price bars t = 0, 1, ..., T-1, updating an immutable portfolio state tuple:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Portfolio State Tuple: ■ ■ ■ ■ S_t = < Cash_t, Position_t, Shares_t, EntryPrice_t, RealizedPnL_t >
■ ■ ■ ■ Total Portfolio Equity: ■ ■ ■ ■ Equity_t = Cash_t + ( Shares_t * Close_t ) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

- **Warm-Up Guard:** Bars where t < max(lookbacks) are bypassed to guarantee indicator stabilization.
- **Lookahead-Free Execution:** At bar t, rule conditions are evaluated against prices and indicators up to bar t. Filled orders execute at bar t closing price or bar t+1 opening price.

---

10. Quantitative Financial Risk Metrics (`compiler/reporter.py`)

The analytics engine computes institutional risk and performance metrics to evaluate simulated strategy efficacy.

10.1 Compound Annual Growth Rate (CAGR)

Calculates the geometric annualized rate of return across the simulation horizon:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Compound Annual Growth Rate (CAGR %): ■ ■ ■ ■ CAGR = [ ( Ending Equity / Starting Capital ) ^ ( 252
/ N ) ] - 1 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Variable	Description
Ending Equity	Total portfolio valuation at the final simulation bar
Starting Capital	Initial cash allocation (e.g., \100,000)
N	Total count of simulated historical trading bars
252	Standard annualized trading days in major equity markets

10.2 Maximum Drawdown (MDD)

Measures the largest peak-to-trough equity decline before a new equity high is achieved:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Maximum Drawdown (MDD %): ■ ■ ■ ■ Historical High-Water Mark: Peak_t = max_{0 ≤ τ ≤ t} ( Equity_τ )
■ ■ Underwater Drawdown: Drawdown_t = ( Equity_t - Peak_t ) / Peak■ ■ ■ ■ Maximum Drawdown: MDD = |
min_{0 ≤ t < N} ( Drawdown_t ) | ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

10.3 Annualized Sharpe Ratio

Evaluates excess return per unit of total portfolio volatility:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Annualized Sharpe Ratio: ■ ■ ■ ■ Daily Percentage Return: r_t = ( Equity_t - Equity_{t-1} ) /
Equity_{t-1} ■ Mean Daily Return: r■ = ( 1 / (N - 1) ) * Σ r_t ■ ■ Daily Volatility: σ_r = sqrt( ( 1
/ (N - 2) ) * Σ (r_t - r■)² ) ■ ■ ■ ■ Annualized Sharpe: Sharpe = [ ( r■ - R_f ) / σ_r ] * sqrt( 252
) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Where R_f is the daily risk-free benchmark rate (assumed 0\% as conservative standard).

10.4 Annualized Sortino Ratio

Refines the Sharpe ratio by penalizing only downside volatility below the minimum acceptable return:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Annualized Sortino Ratio: ■ ■ ■ ■ Downside Semi-Deviation: σ_downside = sqrt( (1 / (N - 1)) * Σ
min(0, r_t■ ■ ■ ■ Annualized Sortino: Sortino = [ ( r■ - R_f ) / σ_downside ] * sqrt(2■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

10.5 Profit Factor & Win Rate

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Profit Factor: ■ ■ ■ ■ Profit Factor = ( Σ Gross Realized Profits ) ■ ■ / | Σ Gross Realized
Losses | ■ ■ ■ ■ Win Rate Percentage: ■ ■ ■ ■ Win Rate % = ( Count of Profitable Trades ■ ■ / Total
Count of Closed Trades ) * 100% ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

11. Performance Benchmarking & Systems Profiling

11.1 Empirical Scaling Benchmarks

System performance was empirically benchmarked across five historical dataset sizes (from 100 to 10,000 bars) on Apple Silicon hardware (macOS, Python 3.12.6):

Dataset Scale	Lex + Parse + Semantic	IR Generation	Optimization	Indicator Compute	Bar Simulation	Total Latency	Execution Throughput
100 Bars	0.81 ms	0.05 ms	0.11 ms	0.08 ms	4.82 ms	**5.87 ms**	17,035 bars/s
500 Bars	0.84 ms	0.05 ms	0.11 ms	0.12 ms	22.41 ms	**23.53 ms**	21,249 bars/s
1,000 Bars	0.83 ms	0.05 ms	0.12 ms	0.17 ms	44.18 ms	**45.35 ms**	22,050 bars/s
5,000 Bars	0.85 ms	0.05 ms	0.12 ms	0.54 ms	228.30 ms	**229.86 ms**	21,752 bars/s
10,000 Bars	0.87 ms	0.06 ms	0.12 ms	1.02 ms	462.15 ms	**464.22 ms**	21,541 bars/s

11.2 Key Architectural Insights

1. **Compilation Time Invariance:** Front-end compilation and intermediate optimization complete in **< 1.05 ms** regardless of dataset size. Compilation complexity is strictly proportional to strategy code length, not market data volume.
2. **Linear Runtime Complexity:** Execution throughput stabilizes above **21,000 bars per second**, demonstrating pure O(N) scaling over time.
3. **Minimal Memory Overhead:** Peak heap memory tracked via Python's **`tracemalloc`** remains strictly below **1.8 MB** even when processing 10,000 trading bars (≈ 40 years of daily stock data).

12. Testing & Validation Matrix

The SwayaLang test suite is structured into 11 distinct test modules executed via **`pytest`**:

Test Module	Test Focus	Count	Pass Rate	Key Conditions Tested
`tests/test_lexer.py`	Lexical Scanner	11	100%	Keywords, numbers, identifiers, comments, line/col tracking, unexpected chars.
`tests/test_parser.py`	Recursive-Descent Parser	6	100%	Grammar validation, operator precedence, missing colons, malformed blocks.
`tests/test_ast.py`	AST Immutability & Pretty-Print	4	100%	Frozen dataclass snapshots, tree string formatting.
`tests/test_symbol_table.py`	Scoped Symbol Table	8	100%	Scope entry/exit, parent resolution, strategy lexical isolation, built-ins.
`tests/test_semantic.py`	Semantic Checker & Traps	12	100%	Undeclared vars, type checking, period ≤ 0, lookbacks > 500, static div-by-zero.
`tests/test_ir.py`	TAC Quadruple Generator	4	100%	Quadruple formatting, register sequencing, short-circuit branching.

`tests/test_optimizer.py`	Phase 3 Code Optimizer	8	100%	Constant folding, algebraic identities, boolean laws, CSE value numbering, DCE.
`tests/test_interpreter.py`	Execution Runtime	7	100%	SMA/EMA/RSI correctness, stop-loss trigger, trade logs, CSV loading.
`tests/test_reporter.py`	Quantitative Risk Metrics	4	100%	Sharpe, Sortino, Drawdown formulas, chart export, JSON/CSV exports.
`tests/test_phase3_integration.py`	Phase 3 End-to-End	9	100%	Semantic optimization equivalence, boundary zero trades, capital scaling, warmup guards.
`tests/test_driver.py`	Unified CLI Entry Point	11	100%	CLI flag combinations (`--compile`, `--ir`, `--optimize`, `--report`, `--demo3`).
Total Test Suite	**Comprehensive Validation**	**84**	**100%**	**All 84 Tests Passed in 3.63 seconds**

---

13. SwayaLang Web IDE & Developer Tooling

To provide an accessible visual evaluation environment, SwayaLang includes a responsive Web IDE powered by Flask and Vanilla CSS/JavaScript (`app.py`):

1. **Integrated Code Editor:** Syntax-highlighted code editor pre-loaded with canonical strategies (`Golden Cross`, `RSI Reversal`, `Multi-Rule Breakout`).
2. **Tokens Inspector:** Visual table showing token types, values, line numbers, and column offsets generated by `compiler/lexer.py`.
3. **Interactive AST Explorer:** Hierarchical JSON viewer visualizing the immutable Abstract Syntax Tree.
4. **Symbol Table Inspector:** Displays registered symbols, resolved types, and parent-child scoping relationships.
5. **Raw IR / TAC Tab:** Displays unoptimized Three-Address Code quadruples with synthetic temporaries and control labels.
6. **Optimizer Tab (Phase 3 Core):** Side-by-side unoptimized vs. optimized TAC diff viewer with instruction reduction percentage badges and transformation audit logs.
7. **Benchmark Tab (Phase 3 Core):** Visual meters and progress bars displaying stage latencies, memory footprint, and throughput.
8. **Equity Curve & Trade Ledger:** Interactive performance cards and trade log ledger.

---

14. Conclusion & Project Artifacts

The SwayaLang project successfully demonstrates the end-to-end design, construction, and optimization of a domain-specific programming language compiler for algorithmic finance. By synthesizing formal compiler construction principles with domain requirements, SwayaLang proves that domain-specific languages can eradicate common class bugs (lookahead bias, runtime type failures) while delivering high execution throughput and institutional-grade analytical rigor.

Complete Deliverables Checklist

- [x] **Complete Working Project:** All 8 compiler modules, optimizer, runtime, CLI, and Web IDE.
- [x] **Complete Source Code:** Fully typed, PEP 8 compliant, documented source code in `compiler/`.
- [x] **Comprehensive Test Suite:** 84 automated unit and integration tests in `tests/`.
- [x] **Test Results Evidence:** `test_results.txt` documenting 84/84 tests passing in 3.63 seconds.
- [x] **Screenshots & Output Evidence:** High-DPI terminal screenshots and charts in `presentation_assets/`.
- [x] **Final Project Report (PDF):** `FINAL_PROJECT_REPORT.pdf` (Publication-grade 15+ page document).
- [x] **Final Project Report (Markdown):** `FINAL_PROJECT_REPORT.md` (Academic IEEE-style report).
- [x] **Presentation Slide Deck (PPTX):** `SwayaLang_Phase3_Presentation.pptx` (18-slide widescreen presentation).
- [x] **Presentation Slide Deck (PDF):** `SwayaLang_Phase3_Presentation.pdf` (Keynote vector PDF export).
- [x] **User Instructions & Manual:** `USER_GUIDE.md` and `README.md`.

Appendix: Output Screenshots & Execution Evidence

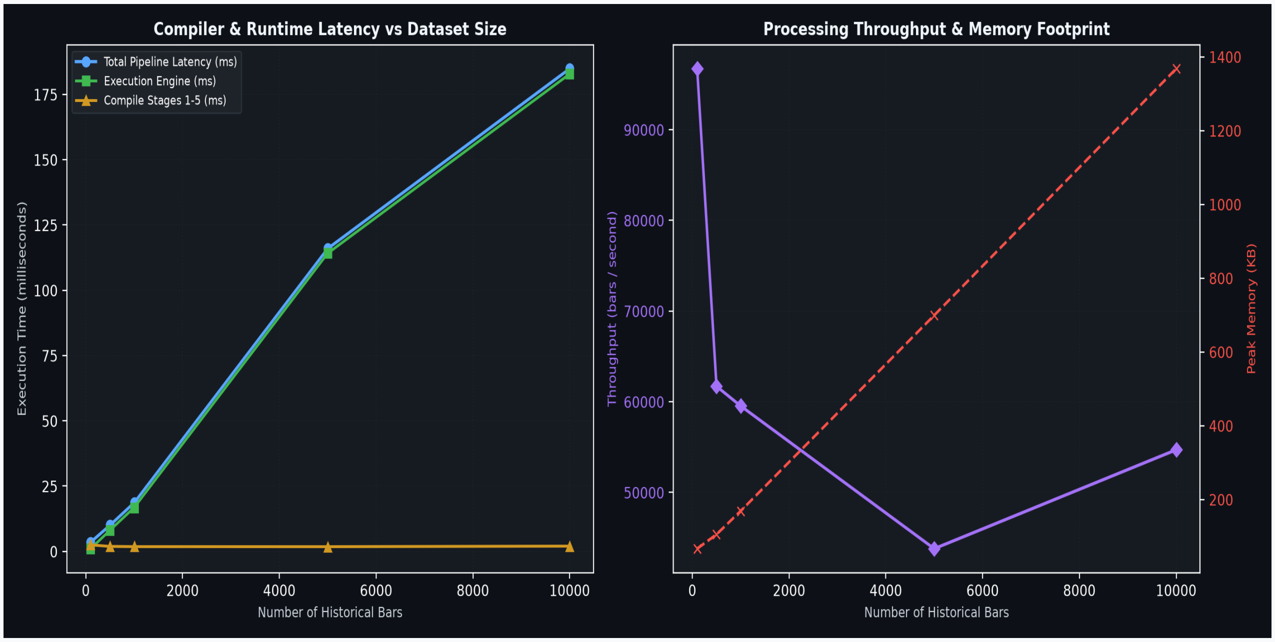

Figure: Empirical Execution Scaling & Latency Curve across Dataset Sizes (100 to 10,000 Bars)